

UNIVERSIDADE FEDERAL
DE SANTA CATARINA

UNIVERSIDADE FEDERAL DE SANTA CATARINA

Departamento de Automação e Sistemas

DAS-5103 - Cálculo Numérico para Controle e Automação

Trabalho Prático 1

João Victor Barbosa Teixeira Bouças

Matrícula 25103276

Florianópolis

2026

1. Introdução

Este relatório trata de três classes de problemas numéricos: a determinação de uma raiz escalar, a solução de um sistema linear e a solução de um sistema não linear. Em cada caso foram implementados em Python os algoritmos especificados no enunciado, sem recorrer a rotinas prontas de busca de raízes ou de solução de sistemas como procedimento principal.

Além dos valores finais, foram examinados os resíduos, o número de iterações e as trajetórias dos iterandos. Os critérios de parada não foram escolhidos livremente: para a Questão 1, adotou-se |f(theta_k)| < 10^-5; para a Questão 3, ||F(x_k)||_2 <= 10^-5; em ambos os casos o limite foi de 100 iterações.

2. Questão 1 - Equação não linear

2.1 Formulação e métodos

Foi resolvida f(theta) = (100²/9,8)[sin(theta) + 0,5 sin(2 theta)] - 1000 = 0, no domínio 0 <= theta <= pi/2. As avaliações iniciais foram f(0,5) = -81,468335706 e f(1,5) = 89,852031259. Assim, o intervalo dos métodos de quebra possui mudança de sinal e está no domínio de interesse.

Aplicaram-se bisseção, falsa posição, Newton e secante com os pontos prescritos. Bisseção preserva o intervalo de sinais opostos e reduz sua largura pela metade. Falsa posição conserva essa propriedade usando a interseção da secante dos extremos. Newton usou a derivada analítica f'(theta) = (100²/9,8)[cos(theta) + cos(2 theta)], enquanto a secante estimou a inclinação com dois iterandos anteriores.

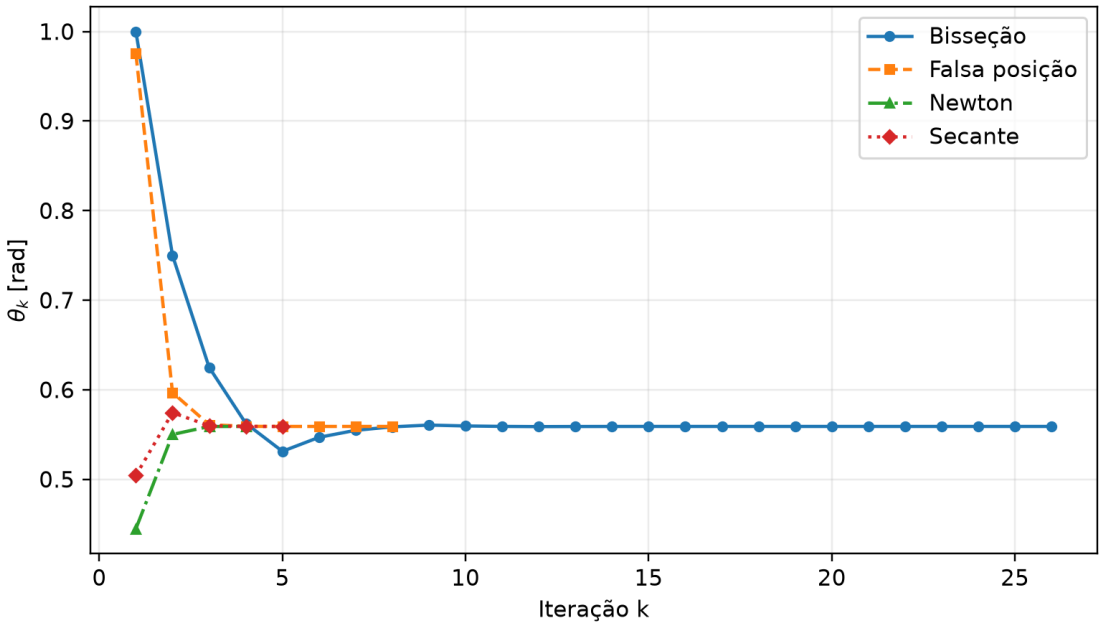


Figura 1 - Trajetória de theta_k para os quatro métodos da Questão 1.

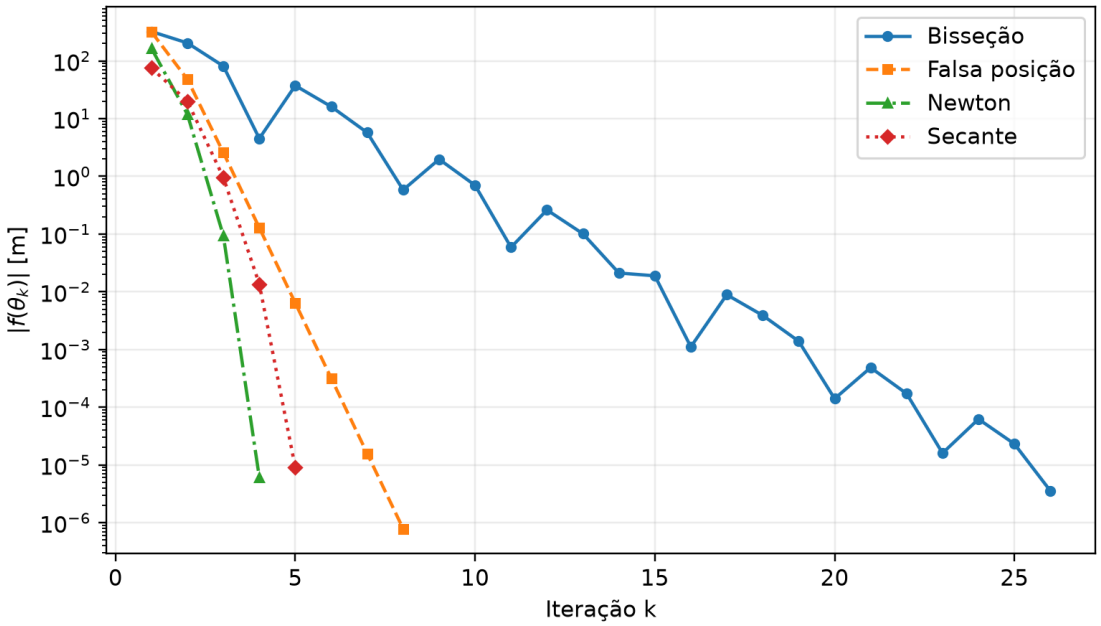


Figura 2 - Módulo do resíduo por iteração na Questão 1.

2.2 Resultados e análise

Tabela 1 - Comparação final dos métodos para a Questão 1.

Método	theta [rad]	theta [graus]	Iterações	|f(theta)|	Situação
Bisseção	0,559037104	32,030466665	26	3,549 x 10^-6	Convergiu
Falsa posição	0,559037102	32,030466544	8	7,821 x 10^-7	Convergiu
Newton	0,559037097	32,030466245	4	6,058 x 10^-6	Convergiu
Secante	0,559037108	32,030466902	5	8,971 x 10^-6	Convergiu

As quatro aproximações pertencem ao domínio e concordam até as casas apresentadas. A substituição direta em f produz os resíduos da Tabela 1; todas satisfazem o critério determinado no enunciado. A bisseção foi a mais lenta, mas preservou o intervalo válido nas 26 iterações.

Na falsa posição, não ocorreu o risco de uma extremidade permanecer praticamente fixa: a tolerância foi atingida em oito passos. Newton foi o mais rápido, com quatro passos a partir de theta_0 = 0,8; a derivada não se aproximou de zero na trajetória. A secante convergiu em cinco passos e seu denominador não se aproximou de zero. Os dois últimos foram rápidos porque os pontos prescritos estavam em uma região favorável, sem a proteção global dos métodos de quebra.

2.3 Conclusão da questão

O ângulo de lançamento é aproximadamente 0,5590371 rad, ou 32,03047 graus. Todos os métodos convergiram; a diferença está no custo iterativo: 26 passos para a bisseção, 8 para a falsa posição, 4 para Newton e 5 para a secante. Neste problema, Newton foi o mais rápido e a bisseção forneceu a trajetória mais previsível.

3. Questão 2 - Sistema de equações lineares

3.1 Sistema e leitura do grafo

O sistema fornecido no enunciado foi escrito na ordem (x1,...,x7)^T. A figura da malha aérea contém arcos curvos que se cruzam e rótulos próximos de outros trechos. A leitura adotada não foi baseada na geometria ou no cruzamento visual das linhas. Cada arco foi identificado pelo rótulo x_i, e seu sentido foi fixado pela tabela e pelas equações do enunciado, que são a especificação formal do problema.

Quadro 1 - Transcrição orientada usada para interpretar a figura da Questão 2.

Variável	Rota lida	Critério de leitura
x1	IGU -> GRU	Rótulo e tabela do enunciado
x2	GRU -> Outros	Rótulo e tabela do enunciado
x3	Outros -> GIG	Rótulo e tabela do enunciado
x4	GIG -> POA	Rótulo e tabela do enunciado
x5	POA -> CGH	Rótulo e tabela do enunciado
x6	SDU -> CGH	Rótulo e tabela do enunciado
x7	CGH -> IGU	Rótulo e tabela do enunciado

Assim, os cruzamentos no desenho são sobreposições de arcos no plano, e não conexões entre aeroportos. A equação -x1 + 2x7 = 0 foi tratada como a relação adicional do enunciado, isto é, x1 = 2x7; ela não foi deduzida de uma leitura espacial da figura.

3.2 Eliminação de Gauss com pivotamento

Foi implementada eliminação de Gauss com pivotamento parcial e retrosubstituição. Em cada coluna, seleciona-se a maior entrada em módulo dentre as linhas disponíveis. Isso evita divisão por pivôs nulos e reduz a exposição a arredondamentos quando há pivôs pequenos. Nas três primeiras colunas houve empate de módulo igual a um; a primeira linha candidata foi mantida, logo não houve troca de linhas.

A matriz aumentada inicial é apresentada abaixo. Em cada etapa, a coluna de pivô, a linha escolhida, o multiplicador e a operação elementar são mostrados antes da matriz resultante.

Matriz aumentada inicial [A | b].

-1	0	0	0	0	0	2	0
1	-1	0	0	0	0	0	-130
0	1	-1	0	0	0	0	102
0	0	1	-1	0	0	0	22
0	0	0	1	-1	0	0	-1
0	0	0	0	0	1	0	29
0	0	0	0	1	1	-1	38

Etapa 1. Coluna de pivô: 1. A linha 1 foi escolhida, com a11 = -1; ela empata em módulo com a linha 2, portanto não houve troca. O único multiplicador não nulo é m21 = 1/(-1) = -1 e a operação é L2 <- L2 - m21 L1 = L2 + L1.

Matriz aumentada após a Etapa 1.

-1	0	0	0	0	0	2	0
0	-1	0	0	0	0	2	-130
0	1	-1	0	0	0	0	102
0	0	1	-1	0	0	0	22
0	0	0	1	-1	0	0	-1
0	0	0	0	0	1	0	29
0	0	0	0	1	1	-1	38

Etapa 2. Coluna de pivô: 2. A linha 2 foi escolhida, com a22 = -1; ela empata em módulo com a linha 3, sem troca. O multiplicador não nulo é m32 = 1/(-1) = -1 e a operação é L3 <- L3 + L2.

Matriz aumentada após a Etapa 2.

-1	0	0	0	0	0	2	0
0	-1	0	0	0	0	2	-130
0	0	-1	0	0	0	2	-28
0	0	1	-1	0	0	0	22
0	0	0	1	-1	0	0	-1
0	0	0	0	0	1	0	29
0	0	0	0	1	1	-1	38

Etapa 3. Coluna de pivô: 3. A linha 3 foi escolhida, com a33 = -1; ela empata em módulo com a linha 4, sem troca. O multiplicador não nulo é m43 = 1/(-1) = -1 e a operação é L4 <- L4 + L3.

Matriz aumentada após a Etapa 3. As etapas restantes e a retrosubstituição geram x = (4, 134, 32, 10, 11, 29, 2)^T.

-1	0	0	0	0	0	2	0
0	-1	0	0	0	0	2	-130
0	0	-1	0	0	0	2	-28
0	0	0	-1	0	0	2	-6
0	0	0	1	-1	0	0	-1
0	0	0	0	0	1	0	29
0	0	0	0	1	1	-1	38

Tabela 2 - Quantidade de voos diários nas rotas desconhecidas.

Variável	Voo	Voos por dia
x1	IGU -> GRU	4
x2	GRU -> Outros	134
x3	Outros -> GIG	32
x4	GIG -> POA	10
x5	POA -> CGH	11
x6	SDU -> CGH	29
x7	CGH -> IGU	2

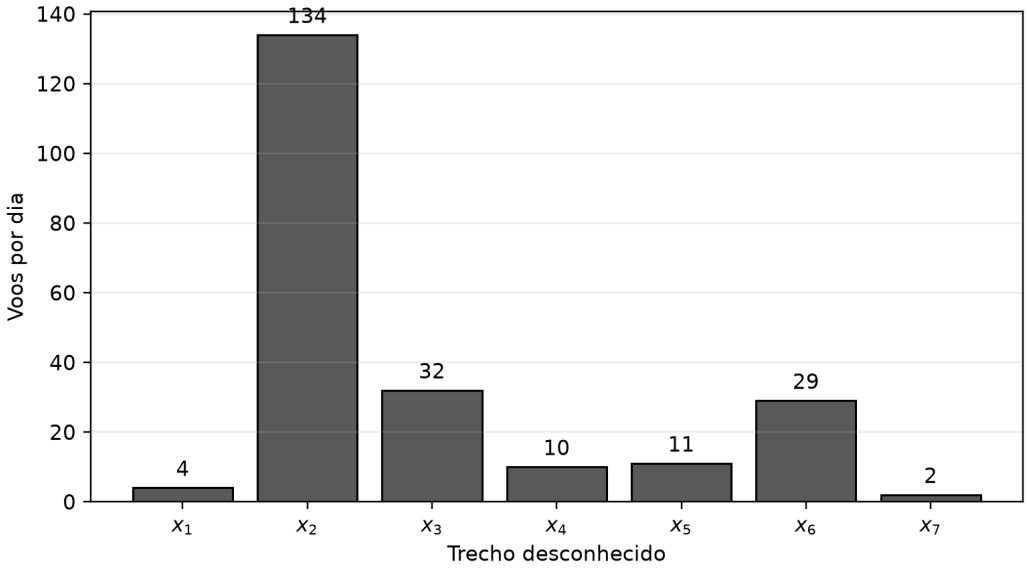


Figura 3 - Fluxos desconhecidos obtidos para a malha aérea.

A substituição independente forneceu Ax-b = (0,0,0,0,0,0,0)^T e ||Ax-b||_2 = 0 na aritmética de ponto flutuante usada. A relação adicional também é satisfeita: x1 = 4 = 2 x 2 = 2x7. Como diagnóstico, kappa_2(A) = 25,433, valor moderado para esta matriz.

3.3 Conclusão da questão

O sistema é consistente e a solução satisfaz todas as equações de conservação e a relação adicional. O residual nulo confirma a substituição numérica. Embora não tenha havido permuta nas primeiras etapas, o pivotamento continua sendo necessário: em outra ordenação ou com pivôs pequenos, ele reduz o risco de uma divisão numericamente insegura.

4. Questão 3 - Sistema de equações não lineares

4.1 Formulação e implementação

O vetor foi mantido na ordem x = [Delta p, Tin, Tout, mu_med]^T. A cada passo de Newton-Raphson, resolveu-se J(x_k) Delta x_k = -F(x_k) por eliminação de Gauss com pivotamento parcial e atualizou-se x_(k+1) = x_k + Delta x_k. A inversa de J não foi formada explicitamente.

Foram investigados os riscos de Jacobiana singular, crescimento do resíduo, iterados não finitos e sensibilidade ao chute inicial. Não se aplicou amortecimento, retrocesso ou troca de método. Os dois casos convergiram antes do limite de 100 passos.

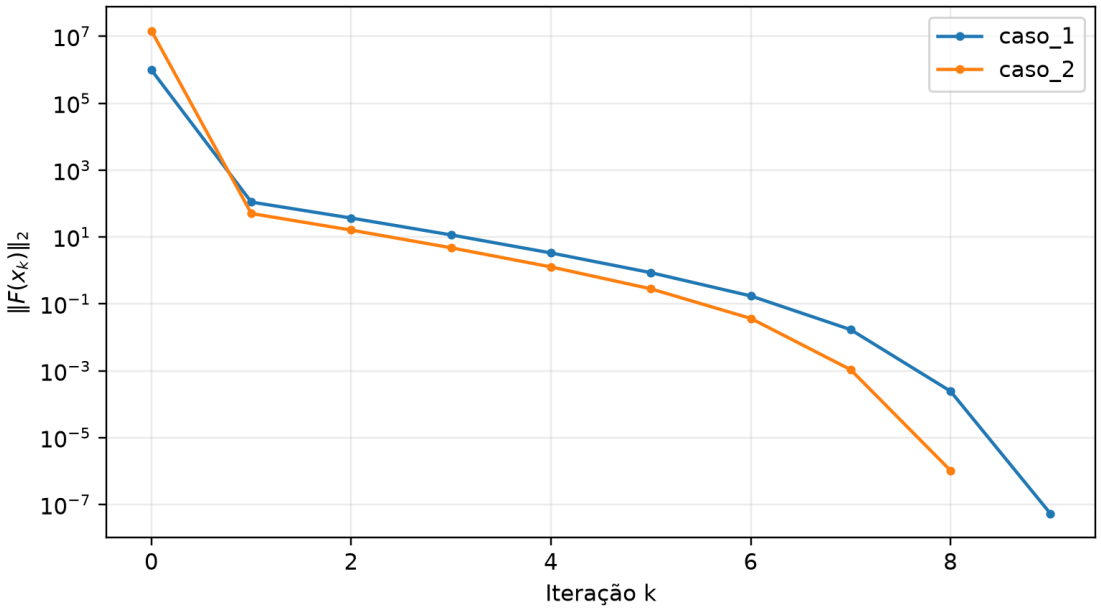


Figura 4 - Norma do resíduo ao longo das iterações de Newton-Raphson.

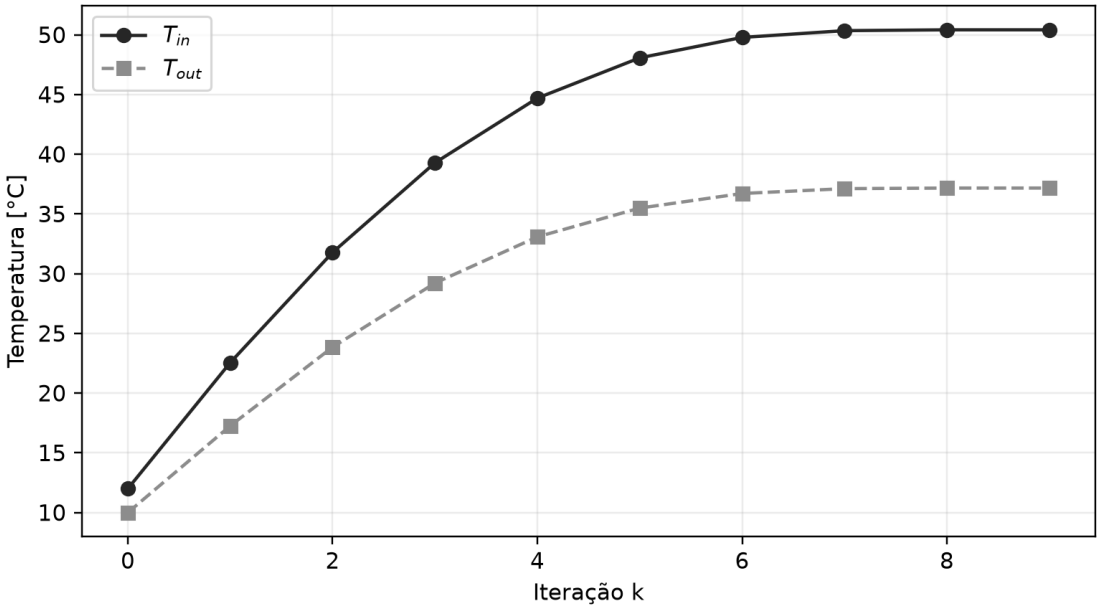


Figura 5 - Trajetória das temperaturas para o Caso 1.

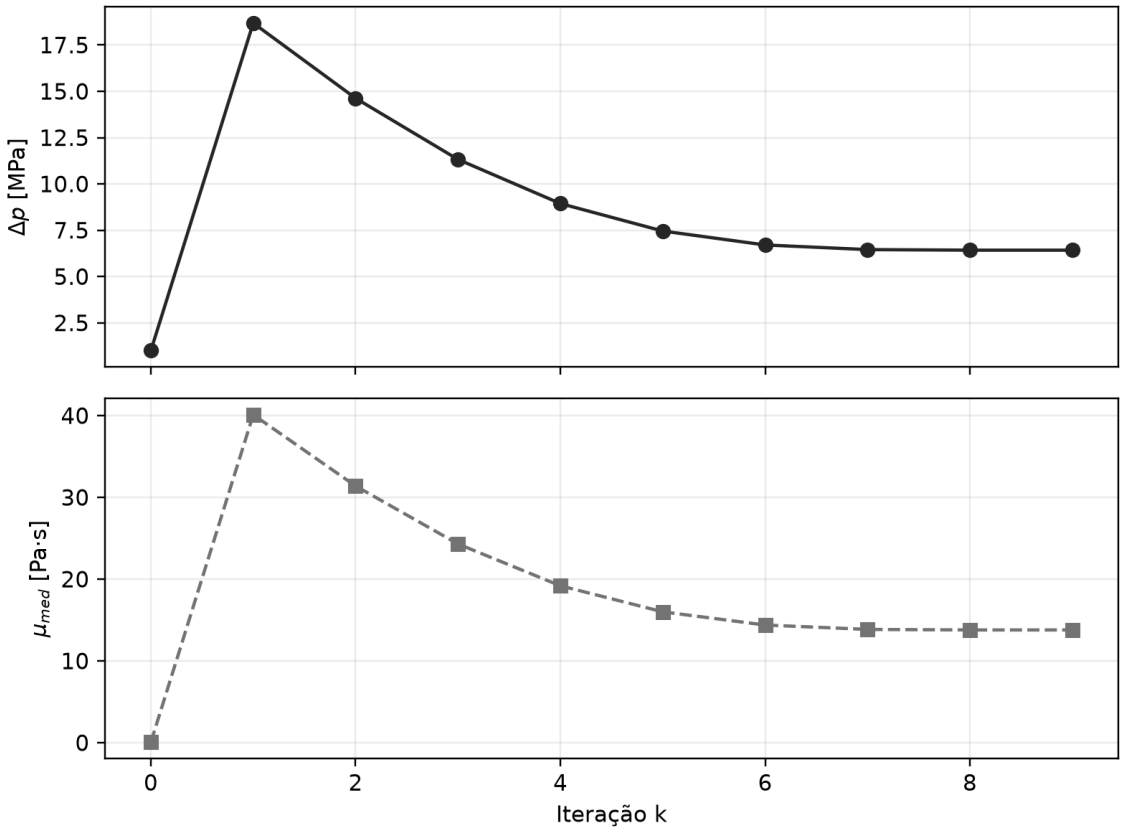


Figura 6 - Trajetória de Delta p e mu_med para o Caso 1.

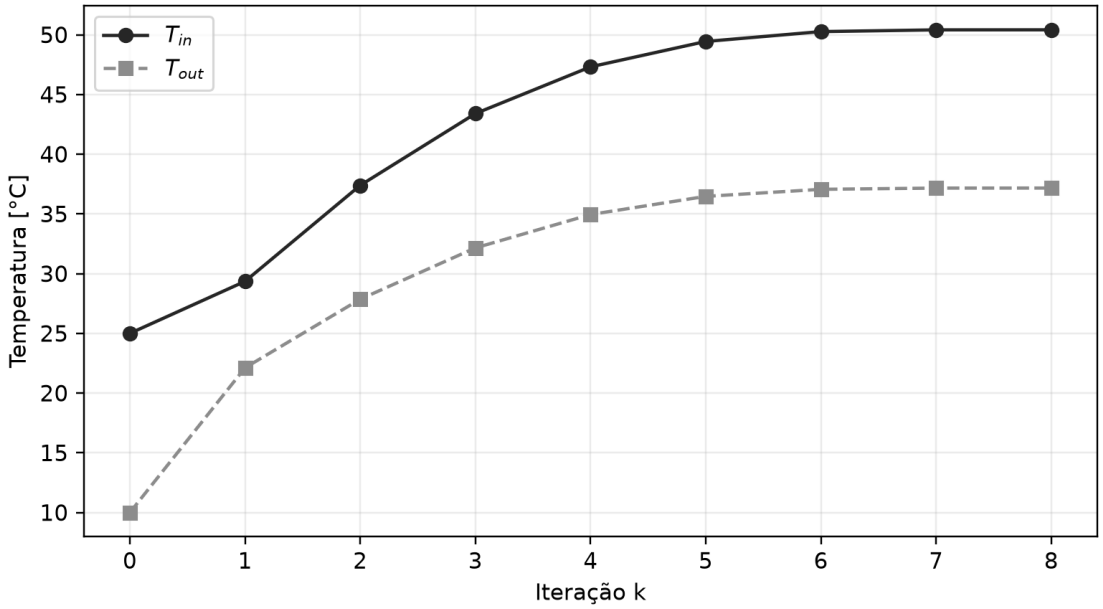


Figura 7 - Trajetória das temperaturas para o Caso 2.

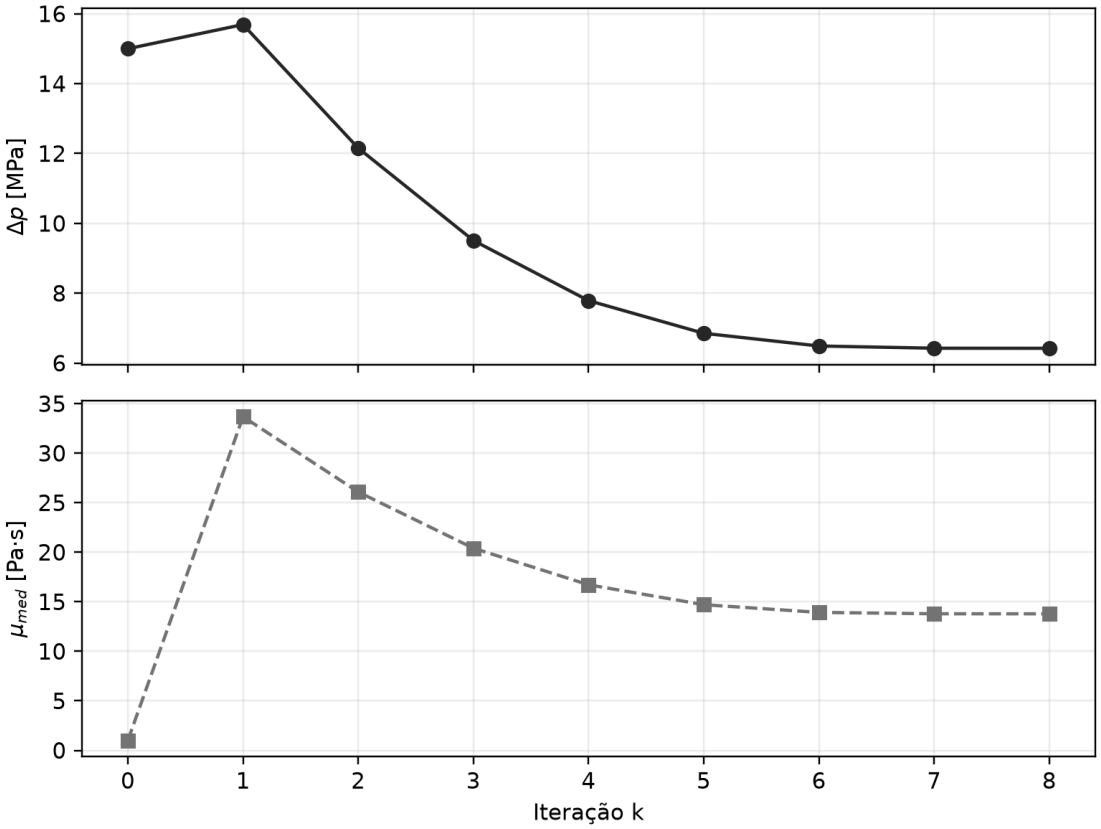


Figura 8 - Trajetória de Delta p e mu_med para o Caso 2.

Tabela 3 - Resultado de Newton-Raphson para os dois chutes prescritos.

Chute	Delta p [MPa]	Tin [°C]	Tout [°C]	mu_med [Pa.s]	Iterações	Situação
Caso 1	6,423854	50,417201	37,168916	13,787461	9	Convergiu
Caso 2	6,423856	50,417197	37,168913	13,787465	8	Convergiu

No Caso 1, a norma inicial foi 9,772 x 10^5 e decresceu para 5,372 x 10^-8 em nove atualizações. No Caso 2, a norma inicial foi 1,457 x 10^7 e atingiu 1,038 x 10^-6 em oito atualizações. As duas normas atendem à tolerância. O número de condição 2 da Jacobiana no final ficou próximo de 7,49 x 10^6. Esse diagnóstico é elevado pela diferença de escala entre as coordenadas e as equações; ele não substituiu o critério de parada. Os resíduos continuaram decrescendo e não houve Jacobiana singular, oscilação ou iterado não finito.

4.2 Análise física e conclusão da questão

Os dois chutes convergiram para a mesma solução dentro da tolerância. Tin aproximadamente 50,42 °C é maior que Tout aproximadamente 37,17 °C, coerente com a perda térmica ao longo do duto para o ambiente a 4 °C. A relação exponencial produz mu_med aproximadamente 13,79 Pa.s nessa faixa e resulta em Delta p aproximadamente 6,424 MPa. O balanço econômico e as outras equações são satisfeitos pelos resíduos finais, sendo a solução compatível com o custo diário ótimo de US$ 6100 estabelecido no problema.

5. Considerações finais

Na equação escalar, os métodos de quebra foram previsíveis por preservarem um intervalo válido, enquanto Newton e secante foram mais rápidos para os pontos iniciais fornecidos. No sistema linear, a eliminação de Gauss com pivotamento produziu uma solução com residual nulo e tornou explícitas as operações de redução. No sistema não linear, Newton-Raphson convergiu a partir dos dois chutes sem amortecimento. Em todos os casos, o residual exigido pelo enunciado foi o critério principal de aceitação.

Referências

COSTA, Lucas Kuhnen; CAMPONOGARA, Eduardo. Trabalho Prático 1. DAS-5103 - Cálculo Numérico para Controle e Automação, UFSC, setembro de 2026.

CAMPONOGARA, Eduardo; CASTELAN NETO, Eugênio de Bona. Cálculo Numérico para Controle e Automação. UFSC, versão preliminar, agosto de 2008.

Apêndice D - Tabelas completas de iteração

As tabelas deste apêndice registram todos os iterandos usados para produzir as figuras e os resultados finais. Os valores são gerados diretamente pelas rotinas apresentadas nos apêndices seguintes.

Tabela D.1 - Histórico completo do método da Bisseção na Questão 1.

k	theta_k [rad]	f(theta_k)	|f(theta_k)|
1	1.000000000	3.225711206e+02	3.225711206e+02
2	0.750000000	2.044757687e+02	2.044757687e+02
3	0.625000000	8.121385981e+01	8.121385981e+01
4	0.562500000	4.527010802e+00	4.527010802e+00
5	0.531250000	-3.734803837e+01	3.734803837e+01
6	0.546875000	-1.612444881e+01	1.612444881e+01
7	0.554687500	-5.726551791e+00	5.726551791e+00
8	0.558593750	-5.816489119e-01	5.816489119e-01
9	0.560546875	1.977221213e+00	1.977221213e+00
10	0.559570312	6.989199879e-01	6.989199879e-01
11	0.559082031	5.891884330e-02	5.891884330e-02
12	0.558837891	-2.612942273e-01	2.612942273e-01
13	0.558959961	-1.011699878e-01	1.011699878e-01
14	0.559020996	-2.112114591e-02	2.112114591e-02
15	0.559051514	1.889995532e-02	1.889995532e-02
16	0.559036255	-1.110318640e-03	1.110318640e-03
17	0.559043884	8.894887504e-03	8.894887504e-03
18	0.559040070	3.892301723e-03	3.892301723e-03
19	0.559038162	1.390995864e-03	1.390995864e-03
20	0.559037209	1.403396923e-04	1.403396923e-04
21	0.559036732	-4.849892040e-04	4.849892040e-04
22	0.559036970	-1.723246883e-04	1.723246883e-04
23	0.559037089	-1.599248117e-05	1.599248117e-05
24	0.559037149	6.217360976e-05	6.217360976e-05
25	0.559037119	2.309056538e-05	2.309056538e-05
26	0.559037104	3.549042390e-06	3.549042390e-06

Tabela D.2 - Histórico completo do método da Falsa posição na Questão 1.

k	theta_k [rad]	f(theta_k)	|f(theta_k)|
1	0.975532111	3.186561141e+02	3.186561141e+02
2	0.596821900	4.783179117e+01	4.783179117e+01
3	0.561004728	2.575760481e+00	2.575760481e+00
4	0.559135072	1.284678378e-01	1.284678378e-01

k	theta_k [rad]	f(theta_k)	|f(theta_k)|
5	0.559041968	6.382296868e-03	6.382296868e-03
6	0.559037343	3.170112666e-04	3.170112666e-04
7	0.559037114	1.574592488e-05	1.574592488e-05
8	0.559037102	7.820985957e-07	7.820985957e-07

Tabela D.3 - Histórico completo do método da Newton na Questão 1.

k	theta_k [rad]	f(theta_k)	|f(theta_k)|
1	0.444733574	-1.647086597e+02	1.647086597e+02
2	0.550057608	-1.187127760e+01	1.187127760e+01
3	0.558965694	-9.365077551e-02	9.365077551e-02
4	0.559037097	-6.057641144e-06	6.057641144e-06

Tabela D.4 - Histórico completo do método da Secante na Questão 1.

k	theta_k [rad]	f(theta_k)	|f(theta_k)|
1	0.504311862	-7.525039477e+01	7.525039477e+01
2	0.574451644	1.993084913e+01	1.993084913e+01
3	0.559764450	9.532227847e-01	9.532227847e-01
4	0.559026730	-1.360109528e-02	1.360109528e-02
5	0.559037108	8.971494481e-06	8.971494481e-06

D.5 Questão 2 - etapas da eliminação

Tabela D.5 - Resumo de todas as etapas de Gauss com pivotamento na Questão 2.

Etapa	Coluna	Linha de pivô	Troca	Multiplicadores não nulos
1	1	1	não	m=-1
2	2	2	não	m=-1
3	3	3	não	m=-1
4	4	4	não	m=-1
5	5	5	não	m=-1
6	6	6	não	m=1

D.6 Questão 3 - histórico do Newton-Raphson

Tabela D.6 - Histórico completo do Caso 1 (convergiu) na Questão 3.

k	Delta p [MPa]	Tin [°C]	Tout [°C]	mu_med [Pa.s]	||F(x_k)||_2
0	1.000000	12.000000	10.000000	0.050000	9.771976e+05
1	18.686490	22.552229	17.257097	40.106650	1.101882e+02
2	14.625316	31.780630	23.851550	31.390187	3.674246e+01
3	11.323033	39.284564	29.213730	24.302526	1.150474e+01
4	8.940701	44.698051	33.082112	19.189347	3.322851e+00
5	7.456774	48.070049	35.491682	16.004408	8.561815e-01
6	6.704588	49.779275	36.713066	14.389999	1.724819e-01
7	6.454822	50.346831	37.118631	13.853927	1.696152e-02
8	6.424307	50.416172	37.168181	13.788434	2.445911e-04
9	6.423854	50.417201	37.168916	13.787461	5.371766e-08

Tabela D.7 - Histórico completo do Caso 2 (convergiu) na Questão 3.

k	Delta p [MPa]	Tin [°C]	Tout [°C]	mu_med [Pa.s]	||F(x_k)||_2
0	15.000000	25.000000	10.000000	1.000000	1.456646e+07
1	15.692030	29.356684	22.119441	33.679666	5.018960e+01
2	12.155553	37.392790	27.861901	26.089356	1.602257e+01
3	9.510421	43.403449	32.157012	20.412132	4.742508e+00
4	7.788372	47.316541	34.953239	16.716115	1.269575e+00
5	6.853098	49.441810	36.471919	14.708744	2.811920e-01
6	6.489727	50.267514	37.061952	13.928845	3.668498e-02
7	6.425848	50.412671	37.165679	13.791740	1.076860e-03
8	6.423856	50.417197	37.168913	13.787465	1.038196e-06

Apêndice A - Código da Questão 1

```
"""Resolve a equação de alcance do foguete pelos quatro métodos solicitados."""

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Callable

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np



TOLERANCE = 1e-5
MAX_ITERATIONS = 100



@dataclass(frozen=True)
class Iteration:
    """Representa um iterando e seu resíduo.

    Args:
        k: Número da iteração.
        theta: Aproximação do ângulo em radianos.
        residual: Valor de f(theta).
    """

    k: int
    theta: float
    residual: float



def f(theta: float) -> float:
    """Calcula a diferença entre o alcance e o alvo.

    Args:
        theta: Ângulo de lançamento em radianos.

    Returns:
        Resíduo do problema de raiz.
    """

    return 100.0**2 / 9.8 * (np.sin(theta) + 0.5 * np.sin(2.0 * theta)) - 1000.0



def df(theta: float) -> float:
    """Calcula a derivada analítica da função de alcance.

    Args:
        theta: Ângulo de lançamento em radianos.

    Returns:
        Derivada de f no ângulo informado.
    """

    return 100.0**2 / 9.8 * (np.cos(theta) + np.cos(2.0 * theta))
```

```
def bisection(function: Callable[[float], float], a: float, b: float) -> list[Iteration]:
    """Aplica bisseção a um intervalo com mudança de sinal.

    Args:
        function: Função escalar cujo zero será aproximado.
        a: Extremidade esquerda do intervalo inicial.
        b: Extremidade direita do intervalo inicial.

    Returns:
        Histórico de iterandos até a parada solicitada.

    Raises:
        ValueError: Se o intervalo inicial não tiver mudança de sinal.
    """

    fa = function(a)
    fb = function(b)
    if fa * fb >= 0.0:
        raise ValueError("O intervalo inicial não contém mudança de sinal.")
    history: list[Iteration] = []
    for k in range(1, MAX_ITERATIONS + 1):
        midpoint = (a + b) / 2.0
        fm = function(midpoint)
        history.append(Iteration(k, midpoint, fm))
        if abs(fm) < TOLERANCE:
            return history
        if fa * fm < 0.0:
            b, fb = midpoint, fm
        else:
            a, fa = midpoint, fm
    return history


def false_position(function: Callable[[float], float], a: float, b: float) -> list[Iteration]:
    """Aplica falsa posição, preservando um intervalo com mudança de sinal.

    Args:
        function: Função escalar cujo zero será aproximado.
        a: Extremidade esquerda do intervalo inicial.
        b: Extremidade direita do intervalo inicial.

    Returns:
        Histórico de iterandos até a parada solicitada.

    Raises:
        ValueError: Se o intervalo inicial não tiver mudança de sinal.
    """

    fa = function(a)
    fb = function(b)
    if fa * fb >= 0.0:
        raise ValueError("O intervalo inicial não contém mudança de sinal.")
    history: list[Iteration] = []
    for k in range(1, MAX_ITERATIONS + 1):
        estimate = a - fa * (b - a) / (fb - fa)
        fe = function(estimate)
        history.append(Iteration(k, estimate, fe))
```

```
        if abs(fe) < TOLERANCE:
            return history
        if fa * fe < 0.0:
            b, fb = estimate, fe
        else:
            a, fa = estimate, fe
    return history


def newton(function: Callable[[float], float], derivative: Callable[[float], float], x0: float) -> list[Iteration]:
    """Aplica o método de Newton com derivada analítica.

    Args:
        function: Função escalar cujo zero será aproximado.
        derivative: Derivada da função escalar.
        x0: Chute inicial.

    Returns:
        Histórico de iterandos até a parada solicitada.

    Raises:
        ZeroDivisionError: Se a derivada se anular em uma iteração.
    """

    x = x0
    history: list[Iteration] = []
    for k in range(1, MAX_ITERATIONS + 1):
        slope = derivative(x)
        if slope == 0.0:
            raise ZeroDivisionError("A derivada anulou durante a iteração de Newton.")
        x = x - function(x) / slope
        fx = function(x)
        history.append(Iteration(k, x, fx))
        if abs(fx) < TOLERANCE:
            return history
    return history


def secant(function: Callable[[float], float], x0: float, x1: float) -> list[Iteration]:
    """Aplica o método da secante com dois iterandos iniciais.

    Args:
        function: Função escalar cujo zero será aproximado.
        x0: Primeiro ponto inicial.
        x1: Segundo ponto inicial.

    Returns:
        Histórico de iterandos até a parada solicitada.

    Raises:
        ZeroDivisionError: Se a diferença de resíduos se anular.
    """

    previous, current = x0, x1
    history: list[Iteration] = []
    for k in range(1, MAX_ITERATIONS + 1):
        f_previous = function(previous)
        f_current = function(current)
        denominator = f_current - f_previous
```

```
        if denominator == 0.0:
            raise ZeroDivisionError("A diferença de resíduos anulou durante a secante.")
        following = current - f_current * (current - previous) / denominator
        f_following = function(following)
        history.append(Iteration(k, following, f_following))
        if abs(f_following) < TOLERANCE:
            return history
        previous, current = current, following
    return history


def save_plots(histories: dict[str, list[Iteration]], output_dir: Path) -> None:
    """Gera os dois gráficos de convergência solicitados.

    Args:
        histories: Históricos indexados pelo nome de cada método.
        output_dir: Diretório de destino das figuras.
    """

    styles = {
        "Bisseção": ("-", "o"),
        "Falsa posição": ("--", "s"),
        "Newton": ("-.", "^"),
        "Secante": (":", "D"),
    }
    plt.rcParams.update({"font.size": 10, "axes.grid": True, "grid.alpha": 0.25})
    figure, axis = plt.subplots(figsize=(7.2, 4.2))
    for name, history in histories.items():
        line, marker = styles[name]
        axis.plot([item.k for item in history], [item.theta for item in history], linestyle=line, marker=marker, markersize=4, label=name)
    axis.set_xlabel("Iteração k")
    axis.set_ylabel(r"$\theta_k$ [rad]")
    axis.legend()
    figure.tight_layout()
    figure.savefig(output_dir / "q1_theta_iteracoes.png", dpi=220)
    plt.close(figure)
    figure, axis = plt.subplots(figsize=(7.2, 4.2))
    for name, history in histories.items():
        line, marker = styles[name]
        axis.semilogy([item.k for item in history], [abs(item.residual) for item in history], linestyle=line, marker=marker, markersize=4, label=name)
    axis.set_xlabel("Iteração k")
    axis.set_ylabel(r"$|f(\theta_k)|$ [m]")
    axis.legend()
    figure.tight_layout()
    figure.savefig(output_dir / "q1_residuo_iteracoes.png", dpi=220)
    plt.close(figure)


def main() -> None:
    """Executa os métodos, salva as figuras e imprime o resumo numérico."""

    output_dir = Path("figures")
    output_dir.mkdir(exist_ok=True)
    histories = {
        "Bisseção": bisection(f, 0.5, 1.5),
        "Falsa posição": false_position(f, 0.5, 1.5),
        "Newton": newton(f, df, 0.8),
        "Secante": secant(f, 0.7, 0.8),
    }
```

```
    save_plots(histories, output_dir)
    print(f"f(0.5) = {f(0.5):.12f}")
    print(f"f(1.5) = {f(1.5):.12f}")
    for name, history in histories.items():
        final = history[-1]
        status = "convergiu" if abs(final.residual) < TOLERANCE else "não convergiu"
        print(f"{name}: theta={final.theta:.12f} rad, graus={np.degrees(final.theta):.12f}, iteracoes={final.k}, residuo={final.residual:.12e}, {status}")


if __name__ == "__main__":
    main()
```

Apêndice B - Código da Questão 2

```
"""Resolve o sistema de conservação de voos por Gauss com pivotamento."""

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np


@dataclass(frozen=True)
class EliminationStep:
    """Registra uma etapa da eliminação de Gauss.

    Args:
        column: Coluna do pivô, em indexação humana.
        pivot_row: Linha escolhida como pivô, em indexação humana.
        swapped: Indica se ocorreu permuta de linhas.
        multipliers: Multiplicadores empregados abaixo do pivô.
        augmented: Matriz aumentada ao término da etapa.
    """

    column: int
    pivot_row: int
    swapped: bool
    multipliers: tuple[float, ...]
    augmented: np.ndarray


def gaussian_elimination_pivoting(matrix: np.ndarray, vector: np.ndarray) -> tuple[np.ndarray, list[EliminationStep]]:
    """Resolve Ax=b por eliminação de Gauss com pivotamento parcial.

    Args:
        matrix: Matriz quadrada dos coeficientes.
        vector: Vetor do lado direito.

    Returns:
        Vetor solução e histórico das etapas de eliminação.

    Raises:
        ValueError: Se for encontrado pivô nulo.
    """

    order = matrix.shape[0]
    augmented = np.column_stack((matrix.astype(float).copy(), vector.astype(float).copy()))
    steps: list[EliminationStep] = []
    for column in range(order - 1):
        pivot_index = column + int(np.argmax(np.abs(augmented[column:, column])))
        if np.isclose(augmented[pivot_index, column], 0.0):
            raise ValueError("O sistema não admite pivô não nulo nesta coluna.")
        swapped = pivot_index != column
        if swapped:
            augmented[[column, pivot_index]] = augmented[[pivot_index, column]]
        multipliers: list[float] = []
```

```
        for row in range(column + 1, order):
            multiplier = augmented[row, column] / augmented[column, column]
            multipliers.append(float(multiplier))
            augmented[row, column:] -= multiplier * augmented[column, column:]
        steps.append(EliminationStep(column + 1, pivot_index + 1, swapped, tuple(multipliers), augmented.copy()))
    if np.isclose(augmented[-1, -2], 0.0):
        raise ValueError("O sistema não admite pivô não nulo na última linha.")
    solution = np.zeros(order)
    for row in range(order - 1, -1, -1):
        solution[row] = (augmented[row, -1] - augmented[row, row + 1:order] @ solution[row + 1:order]) / augmented[row, row]
    return solution, steps


def system() -> tuple[np.ndarray, np.ndarray]:
    """Monta a matriz e o vetor do sistema apresentado no enunciado.

    Returns:
        Matriz A e vetor b da forma Ax=b.
    """

    matrix = np.array([
        [-1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 2.0],
        [1.0, -1.0, 0.0, 0.0, 0.0, 0.0, 0.0],
        [0.0, 1.0, -1.0, 0.0, 0.0, 0.0, 0.0],
        [0.0, 0.0, 1.0, -1.0, 0.0, 0.0, 0.0],
        [0.0, 0.0, 0.0, 1.0, -1.0, 0.0, 0.0],
        [0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0],
        [0.0, 0.0, 0.0, 0.0, 1.0, 1.0, -1.0],
    ])
    vector = np.array([0.0, -130.0, 102.0, 22.0, -1.0, 29.0, 38.0])
    return matrix, vector


def save_bar_chart(solution: np.ndarray, output_dir: Path) -> None:
    """Gera o gráfico de barras dos fluxos de voo desconhecidos.

    Args:
        solution: Vetor solução do sistema de voos.
        output_dir: Diretório de destino da figura.
    """

    figure, axis = plt.subplots(figsize=(7.2, 4.1))
    bars = axis.bar([f"$x_{index}$" for index in range(1, 8)], solution, color="0.35", edgecolor="black")
    axis.set_xlabel("Trecho desconhecido")
    axis.set_ylabel("Voos por dia")
    axis.grid(axis="y", alpha=0.25)
    for bar, value in zip(bars, solution):
        axis.text(bar.get_x() + bar.get_width() / 2.0, bar.get_height() + 2.0, f"{value:.0f}", ha="center", va="bottom")
    figure.tight_layout()
    figure.savefig(output_dir / "q2_voos_por_dia.png", dpi=220)
    plt.close(figure)


def format_matrix(matrix: np.ndarray) -> str:
    """Formata uma matriz para transcrição direta no relatório.

    Args:
        matrix: Matriz numérica a ser exibida.
```

```
    Returns:
        Linhas da matriz formatadas com seis casas decimais.
    """

    return "\n".join(" ".join(f"{value:12.6f}" for value in row) for row in matrix)


def main() -> None:
    """Executa o método, imprime a validação e salva a figura da questão."""

    output_dir = Path("figures")
    output_dir.mkdir(exist_ok=True)
    matrix, vector = system()
    solution, steps = gaussian_elimination_pivoting(matrix, vector)
    residual = matrix @ solution - vector
    save_bar_chart(solution, output_dir)
    print("Solução:", np.array2string(solution, precision=12))
    print("Residual:", np.array2string(residual, precision=12))
    print(f"Norma 2 do residual: {np.linalg.norm(residual):.12e}")
    print(f"Número de condição 2 de A: {np.linalg.cond(matrix):.12f}")
    for step in steps[:3]:
        print(f"\nEtapa {step.column}: linha de pivô {step.pivot_row}, troca={step.swapped}, multiplicadores={step.multipliers}")
        print(format_matrix(step.augmented))


if __name__ == "__main__":
    main()
```

Apêndice C - Código da Questão 3

```
"""Resolve o sistema não linear do oleoduto por Newton-Raphson."""

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np


TOLERANCE = 1e-5
MAX_ITERATIONS = 100


@dataclass(frozen=True)
class NewtonIteration:
    """Armazena um iterando de Newton para o sistema do oleoduto.

    Args:
        k: Número da iteração.
        x: Vetor [Delta p, Tin, Tout, mu_med].
        residual_norm: Norma euclidiana de F(x).
        step_norm: Norma do último passo de Newton.
        condition: Número de condição 2 da Jacobiana no iterando.
    """

    k: int
    x: np.ndarray
    residual_norm: float
    step_norm: float
    condition: float


def residual(x: np.ndarray) -> np.ndarray:
    """Calcula o vetor F(x) do modelo de operação do oleoduto.

    Args:
        x: Vetor [Delta p, Tin, Tout, mu_med].

    Returns:
        Vetor de quatro resíduos do sistema não linear.
    """

    delta_p, tin, tout, mu_med = x
    length = 32000.0
    diameter = 0.6
    area_surface = 60319.0
    mass_flow = 44.0
    ambient_temperature = 4.0
    source_temperature = 10.0
    density = 950.0
    velocity = 0.1638
    heat_capacity = 2100.0
    transfer_coefficient = 0.510
```

```
    pump_efficiency = 0.75
    electricity_cost = 0.17
    heating_cost = 0.05
    mean_temperature = (tin + tout) / 2.0
    return np.array([
        delta_p * 1e6 - 32.0 * mu_med * length * velocity / diameter**2,
        mass_flow * heat_capacity * (tin - tout) - transfer_coefficient * area_surface * ((tin - ambient_temperature + tout - ambient_temperature) / 2.0),
        6100.0 - delta_p * 1e6 * mass_flow / (pump_efficiency * density) / 1000.0 * electricity_cost * 24.0 - mass_flow * heat_capacity * (tin -
source_temperature) / 1000.0 * heating_cost * 24.0,
        mu_med - 1100.0 * np.exp(-0.1 * mean_temperature),
    ])



def jacobian(x: np.ndarray) -> np.ndarray:
    """Calcula a Jacobiana analítica do sistema não linear.

    Args:
        x: Vetor [Delta p, Tin, Tout, mu_med].

    Returns:
        Matriz Jacobiana J(x).
    """

    _, tin, tout, _ = x
    length = 32000.0
    diameter = 0.6
    area_surface = 60319.0
    mass_flow = 44.0
    density = 950.0
    heat_capacity = 2100.0
    transfer_coefficient = 0.510
    pump_efficiency = 0.75
    electricity_cost = 0.17
    heating_cost = 0.05
    velocity = 0.1638
    exponential = np.exp(-0.1 * (tin + tout) / 2.0)
    return np.array([
        [1e6, 0.0, 0.0, -32.0 * length * velocity / diameter**2],
        [0.0, mass_flow * heat_capacity - transfer_coefficient * area_surface / 2.0, -mass_flow * heat_capacity - transfer_coefficient * area_surface / 2.0,
0.0],
        [-1e6 * mass_flow / (pump_efficiency * density) / 1000.0 * electricity_cost * 24.0, -mass_flow * heat_capacity / 1000.0 * heating_cost * 24.0, 0.0,
0.0],
        [0.0, 55.0 * exponential, 55.0 * exponential, 1.0],
    ])



def gaussian_elimination_pivoting(matrix: np.ndarray, vector: np.ndarray) -> np.ndarray:
    """Resolve um sistema linear por Gauss com pivotamento parcial.

    Args:
        matrix: Matriz quadrada do sistema linear.
        vector: Vetor do lado direito.

    Returns:
        Solução do sistema linear.

    Raises:
        ValueError: Se um pivô nulo impedir a solução.
    """

    order = matrix.shape[0]
```

```
    augmented = np.column_stack((matrix.astype(float).copy(), vector.astype(float).copy()))
    for column in range(order - 1):
        pivot = column + int(np.argmax(np.abs(augmented[column:, column])))
        if np.isclose(augmented[pivot, column], 0.0):
            raise ValueError("Jacobiana singular durante a eliminação.")
        if pivot != column:
            augmented[[column, pivot]] = augmented[[pivot, column]]
        for row in range(column + 1, order):
            multiplier = augmented[row, column] / augmented[column, column]
            augmented[row, column:] -= multiplier * augmented[column, column:]
    solution = np.zeros(order)
    for row in range(order - 1, -1, -1):
        if np.isclose(augmented[row, row], 0.0):
            raise ValueError("Jacobiana singular durante a retrosubstituição.")
        solution[row] = (augmented[row, -1] - augmented[row, row + 1:order] @ solution[row + 1:order]) / augmented[row, row]
    return solution


def newton_raphson(initial: np.ndarray) -> tuple[list[NewtonIteration], str]:
    """Executa Newton-Raphson sem amortecimento para um chute inicial.

    Args:
        initial: Chute inicial para [Delta p, Tin, Tout, mu_med].

    Returns:
        Histórico de iterandos e situação da execução.
    """

    x = initial.astype(float).copy()
    initial_residual = residual(x)
    history = [NewtonIteration(0, x.copy(), float(np.linalg.norm(initial_residual)), 0.0, float(np.linalg.cond(jacobian(x))))]
    for k in range(1, MAX_ITERATIONS + 1):
        try:
            step = gaussian_elimination_pivoting(jacobian(x), -residual(x))
        except (ValueError, FloatingPointError, OverflowError):
            return history, "interrompido por Jacobiana singular ou não finita"
        x = x + step
        if not np.all(np.isfinite(x)):
            return history, "interrompido por iterado não finito"
        current_residual = residual(x)
        if not np.all(np.isfinite(current_residual)):
            return history, "interrompido por resíduo não finito"
        history.append(NewtonIteration(k, x.copy(), float(np.linalg.norm(current_residual)), float(np.linalg.norm(step)), float(np.linalg.cond(jacobian(x)))))
        if history[-1].residual_norm <= TOLERANCE:
            return history, "convergiu"
    return history, "não convergiu segundo o critério estabelecido"


def save_plots(histories: dict[str, list[NewtonIteration]], statuses: dict[str, str], output_dir: Path) -> None:
    """Gera os gráficos de trajetória somente para casos convergentes.

    Args:
        histories: Históricos indexados pelo nome de cada caso.
        statuses: Situações finais indexadas pelo nome de cada caso.
        output_dir: Diretório de destino das figuras.
    """

    converged = {name: history for name, history in histories.items() if statuses[name] == "convergiu"}
    for name, history in converged.items():
```

```
        indices = [item.k for item in history]
        vectors = np.array([item.x for item in history])
        figure, axis = plt.subplots(figsize=(7.2, 4.1))
        axis.plot(indices, vectors[:, 1], "-o", color="0.15", label=r"$T_{in}$")
        axis.plot(indices, vectors[:, 2], "--s", color="0.55", label=r"$T_{out}$")
        axis.set_xlabel("Iteração k")
        axis.set_ylabel("Temperatura [°C]")
        axis.grid(alpha=0.25)
        axis.legend()
        figure.tight_layout()
        figure.savefig(output_dir / f"q3_{name}_temperaturas.png", dpi=220)
        plt.close(figure)
        figure, axes = plt.subplots(2, 1, figsize=(7.2, 5.5), sharex=True)
        axes[0].plot(indices, vectors[:, 0], "-o", color="0.15")
        axes[0].set_ylabel(r"$\Delta p$ [MPa]")
        axes[0].grid(alpha=0.25)
        axes[1].plot(indices, vectors[:, 3], "--s", color="0.45")
        axes[1].set_xlabel("Iteração k")
        axes[1].set_ylabel(r"$\mu_{med}$ [Pa·s]")
        axes[1].grid(alpha=0.25)
        figure.tight_layout()
        figure.savefig(output_dir / f"q3_{name}_pressao_viscosidade.png", dpi=220)
        plt.close(figure)
    figure, axis = plt.subplots(figsize=(7.2, 4.1))
    for name, history in histories.items():
        axis.semilogy([item.k for item in history], [max(item.residual_norm, np.finfo(float).tiny) for item in history], "-o", markersize=3, label=name)
    axis.set_xlabel("Iteração k")
    axis.set_ylabel(r"$\|F(x_k)\|_2$")
    axis.grid(alpha=0.25)
    axis.legend()
    figure.tight_layout()
    figure.savefig(output_dir / "q3_residuo_iteracoes.png", dpi=220)
    plt.close(figure)


def main() -> None:
    """Executa os dois casos, salva os gráficos e imprime as validações."""

    output_dir = Path("figures")
    output_dir.mkdir(exist_ok=True)
    cases = {
        "caso_1": np.array([1.0, 12.0, 10.0, 0.05]),
        "caso_2": np.array([15.0, 25.0, 10.0, 1.0]),
    }
    results = {name: newton_raphson(initial) for name, initial in cases.items()}
    histories = {name: result[0] for name, result in results.items()}
    statuses = {name: result[1] for name, result in results.items()}
    save_plots(histories, statuses, output_dir)
    for name, history in histories.items():
        final = history[-1]
        print(f"{name}: {statuses[name]}")
        print("x final:", np.array2string(final.x, precision=12))
        print("F(x final):", np.array2string(residual(final.x), precision=12))
        print(f"norma final: {final.residual_norm:.12e}, iteracoes: {final.k}, condicao J: {final.condition:.12e}")


if __name__ == "__main__":
    main()
```